

Metody Obliczeniowe w Nauce i Technice

Laboratorium 7: Rozwiązywanie układów równań liniowych metodami bezpośrednimi

Autor: Wiktor Trybus | Grupa laboratoryjna: 4 | Data: 12.05.2026

1. Dane techniczne

Obliczenia oraz generację wykresów zrealizowano przy użyciu następujących narzędzi:

- **Język programowania:** Python 3.13.3
- **Główne biblioteki:** NumPy, Matplotlib, Pandas
- **Precyzja obliczeń:** 32-bitowa oraz 64-bitowa precyzja zmiennoprzecinkowa (float32 oraz float64 z biblioteki NumPy)
- **Środowisko i sprzęt:** - System operacyjny: Windows 11 x64
 - Procesor: Intel Core i7-12700H; 2,30GHz - 4,70GHz

2. Analiza przypadku

W zadaniu na te laboratoria zdefiniowane zostały trzy macierze kwadratowe o wymiarach $n \times n$, które od teraz będę nazywał kolejno A_I , A_{II} oraz A_{III} :

$$A_I = [a_{ij}]_{n \times n} \quad \text{taka, że} \quad a_{ij} = \begin{cases} 1 & \text{dla } i = 1 \\ & j \in \{1, 2, \dots, n\} \\ \frac{1}{i+j-1} & \text{dla } i \neq 1 \\ & j \in \{1, 2, \dots, n\} \end{cases}$$

$$A_{II} = [a_{ij}]_{n \times n} \quad \text{taka, że} \quad \begin{cases} a_{ij} = \frac{2i}{j} & \text{dla } j \geq i \\ a_{ij} = a_{ji} & \text{dla } j < i \end{cases} \quad i, j \in \{1, 2, \dots, n\}$$

$$A_{III} = [a_{i,j}]_{n \times n} \quad \text{taka, że} \quad \begin{cases} a_{i,i} = k \\ a_{i,i+1} = \frac{1}{i+m} \\ a_{i,i-1} = \frac{k}{i+m+1} & \text{dla } i > 1 \\ a_{i,j} = 0 & \text{dla } j < i-1 \quad \text{oraz} \quad j > i+1 \end{cases} \quad i, j \in \{1, 2, \dots, n\}, k = 6, m = 5$$

Postawiona została również definicja macierzy $X_{n \times 1}$, który stanowi rozwiązanie układu równań, który stworzymy przy użyciu zadanych macierzy A_i . Został on zdefiniowany jako n-elementowa permutacja liczb ze zbioru $\{-1, 1\}$:

$$X_{n \times 1} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} \quad \text{gdzie } \forall_{i \in \mathbb{N}} x_i \in \{-1, 1\}$$

Na podstawie wyżej zdefiniowanych macierzy A oraz X możemy wyliczyć macierz B , czyli macierz wyrazów wolnych:

$$B = A \cdot X$$

Teraz mamy już kompletny układ równań. Głównym zadaniem w tym laboratorium jest, aby mając zadane macierze A oraz B w jakiejś konkretnej precyzji zmiennoprzecinkowej (do wyboru `float32` oraz `float64` z biblioteki NumPy) odnaleźć macierz $\bar{X}$, a następnie porównać ją z macierzą X , czyli referencyjną macierzą, która posłużyła do wygenerowania macierzy B .

3. Podstawy teoretyczne

Po odszukaniu macierzy $\bar{X}$ musimy mieć sposób na to, by ocenić jak precyzyjnie udało nam się odszukać rozwiązanie. Aby to zrobić musimy wprowadzić jakąś normę, ja zdecydowałem się na użycie normy maksimum:

$$\|X\|_{\max} = \max_{i \in \{1, 2, \dots, n\}} (|x_i - \bar{x}_i|)$$

Uprzedzając nieco obserwacje z eksperymentów, okaże się, że niektóre układy równań udaje się rozwiązać z dużą precyzją, a niektóre są bardzo problematyczne. Aby zrozumieć dlaczego tak się dzieje posłużymy się pojęciem wskaźnika uwarunkowania macierzy. Jest to wskaźnik opisujący sam problem, nie algorytm, jego wartość dana jest wzorem:

$$\text{cond}(A) = \|A\| \cdot \|A^{-1}\|$$

gdzie $\|\cdot\|$ jest normą spektralną (2-norm)

Jako normę macierzy wybieram normę spektralną, gdyż jest ona rzekomo najpopularniejsza, a także domyślnie używana przez funkcję `numpy.linalg.cond`, której będę używał do obliczenia wskaźnika uwarunkowania. Z tym wskaźnikiem wiąże się jednak pewien problem, a mianowicie odwrócenie macierzy. Jeśli macierz jest źle uwarunkowana, wtedy operacja odwrócenia macierzy również daje wynik obarczony sporym błędem. Nas, w tym laboratorium, interesować będzie jednak w zasadzie rząd tego współczynnika aniżeli dokładna wartość, więc odwracanie macierzy nie będzie stanowić zbyt wielkiego kłopotu.

Kończąc rozważania teoretyczne, warto wspomnieć o samej metodzie rozwiązywania układów równań. Do badań na macierzach A_I oraz A_{II} korzystać będziemy z algorytmu Gaussa z częściowym wyborem elementu podstawowego (*partial pivoting*), który jest standardem. Eksperyment trzeci, na macierzy A_{III} , umożliwił będzie zastosowanie wyspecjalizowanego algorytmu, gdyż macierz ta jest trójdzielna. Dzięki temu, oprócz algorytmu Gaussa, zastosujemy tutaj również algorytm Thomasa.

4. Eksperyment na macierzy A_I

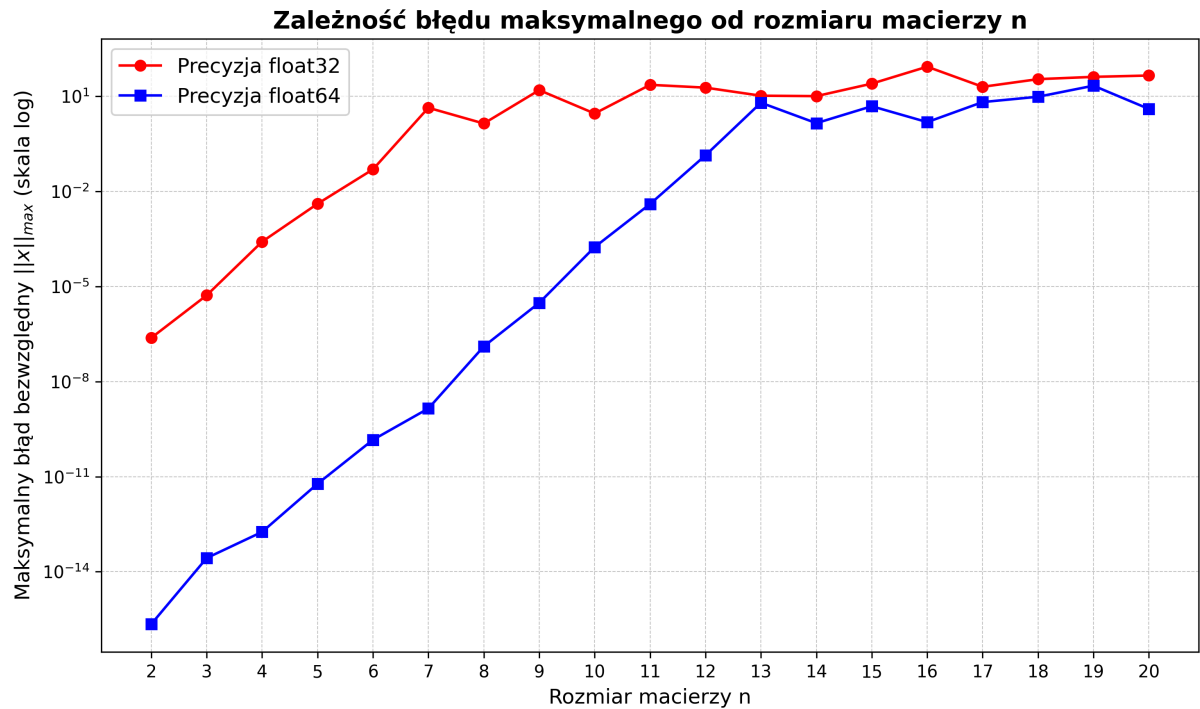
Pierwszy eksperyment polegał na tym, aby obliczyć wektor X jako n -elementową permutację liczb -1 oraz 1 , następnie wyliczyć wektor wyrazów wolnych B , aby finalnie znaleźć wektor $\bar{X}$ stanowiący rozwiązanie układu równań $A_I \cdot \bar{X} = B$. Należało zbadać, przy użyciu ustalonej we wstępie teoretycznym normy, dokładność uzyskanego rozwiązania w zależności od rozmiaru macierzy (n) oraz użytej reprezentacji liczb zmiennoprzecinkowych (`float32`, `float64`).

Zestawienie wyników układu równań macierzy A_I		
	Precyzja float32	Precyzja float64
n	$\ \bar{X}\ _{\max}$	$\ \bar{X}\ _{\max}$
2	2.3842e-07	2.2204e-16
3	5.2452e-06	2.6978e-14
4	2.5749e-04	1.7986e-13
5	4.0412e-03	5.9348e-12
6	4.9290e-02	1.4487e-10
7	4.3252e+00	1.4100e-09
8	1.3784e+00	1.2810e-07
9	1.5591e+01	3.0394e-06
10	2.8484e+00	1.7405e-04
11	2.2907e+01	3.9865e-03
12	1.8665e+01	1.3677e-01
13	1.0387e+01	6.2447e+00
14	1.0080e+01	1.4152e+00
15	2.4967e+01	4.8320e+00
16	8.5437e+01	1.5255e+00
17	1.9838e+01	6.5424e+00
18	3.4593e+01	9.6368e+00
19	4.0839e+01	2.1487e+01
20	4.5035e+01	3.9686e+00

Tabela 1: Wyniki wyznaczone po rozwiązaniu układu równań $A_I \cdot \bar{X} = B$

Tabela 1 prezentuje wyniki pierwszego eksperymentu. Możemy tutaj zauważyć ogromny spadek dokładności dla obu precyzji. Analizując kolumnę z wartościami normy maksymalnej (a więc maksymalną odchyłką od wartości referencyjnej) dla precyzji `float32` zauważamy, że już dla rozmiaru macierzy $n = 7$ nasz błąd wynosi około 4.33. Biorąc pod uwagę fakt, że nasz oryginalny wektor X składał się wyłącznie z wartości -1 lub 1 można uznać, że rozwiązanie tego układu równań metodą Gaussa dla rozmiaru macierzy większego, lub równego $n = 7$, daje zupełnie niemiernodajne wyniki. Podobne wnioski można wysnuć dla podwójnej precyzji (`float64`), natomiast tutaj załamanie następuje po przekroczeniu wartości $n = 12$, tj. wyniki dla rozmiarów większych, lub równych $n = 13$ są niemiernodajne.

Aby lepiej uwidocznic skalę błędu, poniżej znajduje się Rysunek 1 prezentujący wartość normy maksimum dla eksperymentu na macierzy A_I .



Rysunek 1: Wykres normy maksimum, obliczonej dla rozwiązania eksperymentu pierwszego. Skala logarytmiczna.

Powód takiej degradacji precyzji obliczeń jest ściśle powiązany z tzw. współczynnikiem uwarunkowania macierzy. Najlepiej będzie to jednak udowodnić na przykładzie, który stanowi eksperyment drugi, na macierzy A_{II} .

5. Eksperyment na macierzy A_{II}

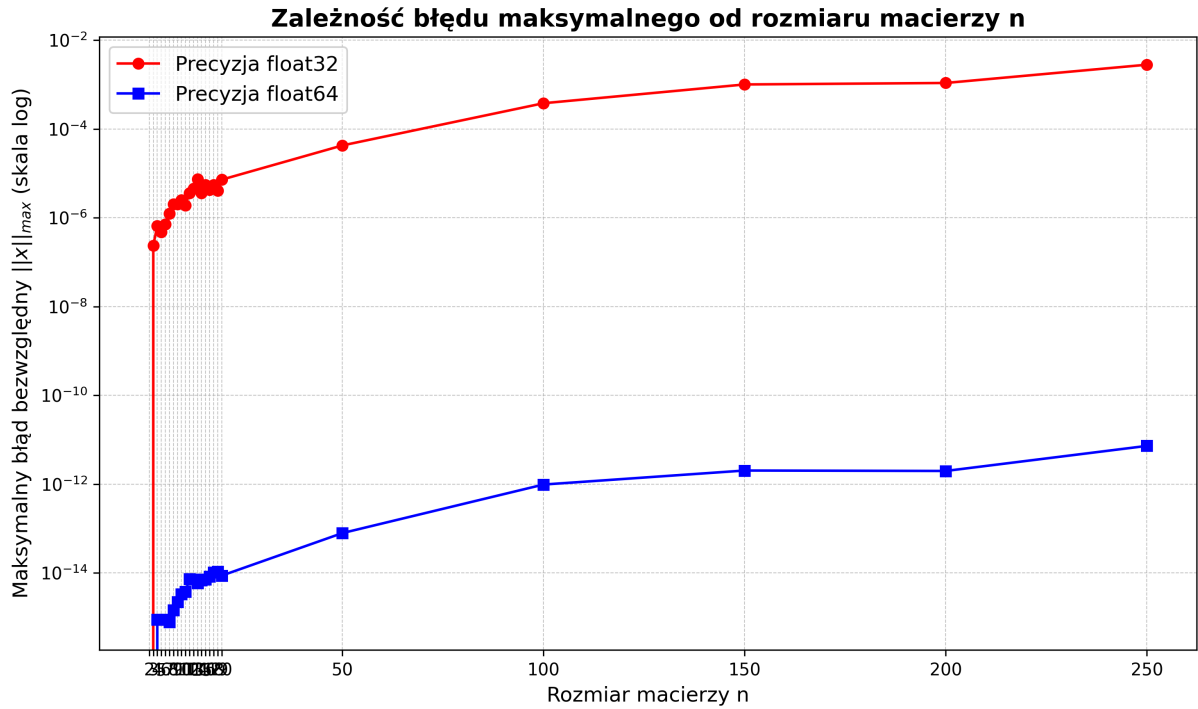
Drugi eksperyment, który należało wykonać w ramach tego laboratorium, to powtórzenie metodyki z eksperymentu pierwszego, lecz na macierzy A_{II} , która została zdefiniowana we wstępie teoretycznym. Poniżej znajduje się tabela, która prezentuje wyniki eksperymentu.

Zestawienie wyników układu równań macierzy A_{II}		
	Precyzja float32	Precyzja float64
n	$\ x\ _{\max}$	$\ x\ _{\max}$
2	0.0000e+00	0.0000e+00
3	2.3842e-07	0.0000e+00
4	6.5565e-07	8.8818e-16
5	4.7684e-07	8.8818e-16
6	7.1526e-07	8.8818e-16
7	1.2517e-06	7.7716e-16
8	2.0266e-06	1.4433e-15
9	2.0266e-06	2.2204e-15
10	2.5034e-06	3.3307e-15
11	1.9073e-06	3.7748e-15
12	3.5763e-06	7.3275e-15
13	4.5896e-06	7.1054e-15
14	7.3910e-06	5.8842e-15
15	3.6359e-06	6.7724e-15
16	5.4836e-06	7.1054e-15
17	4.2319e-06	8.2157e-15
18	5.5432e-06	1.0325e-14
19	4.1127e-06	1.0658e-14
20	7.2122e-06	8.6597e-15
50	4.2558e-05	7.8715e-14
100	3.8016e-04	9.7655e-13
150	1.0071e-03	2.0217e-12
200	1.0879e-03	1.9769e-12
250	2.8057e-03	7.2535e-12

Tabela 2: Wyniki wyznaczone po rozwiązaniu układu równań $A_{II} \cdot X = B$

Tabela ta znacząco różni się od Tabeli 1. W eksperymencie dotyczącym zadania pierwszego, już od rozmiaru $n = 7$ obserwowaliśmy zupełnie niemiarodajne wyniki. Dla macierzy A_{II} okazuje się, że błąd rośnie niezwykle powoli. Dla potwierdzenia tej tezy wygenerowano również serię dodatkowych układów równań dla bardzo dużych macierzy ($n \in \{50, 100, 150, 200, 250\}$), lecz z tymi układami poradził sobie nawet `float32`, uzyskując wynik obciążony błędem rzędu 10^{-3} . Podwójna precyzja poradziła sobie doskonale, generując maksymalną wartość normy maksimum na poziomie $7 \cdot 10^{-12}$.

Analogicznie do pierwszego eksperymentu, dla układów równań zadanych przy pomocy macierzy A_{II} również wygenerowano wykres normy maksimum (Rysunek 2) w zależności od rozmiaru macierzy oraz precyzji. Znajduje się on poniżej.



Rysunek 2: Wykres normy maksimum, obliczonej dla rozwiązania eksperymentu drugiego. Skala logarytmiczna.

5.1. Uwarunkowanie macierzy

Aby odpowiedzieć na pytanie dlaczego obliczenia na macierzy A_I sypią się już przy rozmiarach rzędu $n = 7$, podczas, gdy obliczenia (nawet te wykonane w pojedynczej precyzji) na macierzy A_{II} dają miarodajne wyniki również dla rozmiarów rzędu $n = 250$, musimy rozważyć pojęcie uwarunkowania macierzy.

Uwarunkowanie macierzy to cecha samej macierzy, tj. elementów, które się w niej znajdują. Do opisu uwarunkowania korzysta się z tzw. wskaźnika uwarunkowania macierzy, który, dla przypomnienia, zadany jest wzorem:

$$\text{cond}(A) = \|A\| \cdot \|A^{-1}\|$$

gdzie $\|\cdot\|$ jest normą spektralną (2-norm)

Nie wchodząc tutaj w szczegóły teoretyczne, wskaźnik uwarunkowania zawsze jest większy, lub równy 1. Jeśli jest równy liczbie bliskiej jedynce, macierz jest dobrze uwarunkowana i w efekcie obliczenia na niej dają miarodajny wynik, nieobarczony zbyt dużym błędem. Im większy współczynnik uwarunkowania, tym mniej dokładne wyniki dają obliczenia.

Aby zbadać w jaki sposób zachowuje się współczynnik uwarunkowania macierzy, przeprowadzono eksperyment obliczający ten współczynnik przy użyciu `numpy.linalg.cond` w zależności od rozmiaru macierzy. Zestawienie wartości współczynników dla macierzy A_I oraz macierzy A_{II} prezentuje poniższa tabela.

	Współczynnik uwarunkowania macierzy			
	Macierz A_I		Macierz A_{II}	
n	precyzja float32	precyzja float64	precyzja float32	precyzja float64
2	1.4096e+01	1.4096e+01	3.0000e+00	3.0000e+00
3	4.8292e+02	4.8292e+02	6.6633e+00	6.6633e+00
4	1.7033e+04	1.7033e+04	1.2206e+01	1.2206e+01
5	5.9238e+05	5.9188e+05	1.9655e+01	1.9655e+01
6	2.0957e+07	2.0379e+07	2.9040e+01	2.9040e+01
7	1.3341e+09	6.9805e+08	4.0387e+01	4.0387e+01
8	3.0329e+09	2.3840e+10	5.3717e+01	5.3717e+01
9	8.6613e+09	8.1269e+11	6.9047e+01	6.9047e+01
10	3.6261e+09	2.7673e+13	8.6389e+01	8.6389e+01
11	1.9588e+09	9.4194e+14	1.0576e+02	1.0576e+02
12	2.3648e+09	3.1350e+16	1.2716e+02	1.2716e+02
13	3.7339e+09	5.7621e+17	1.5060e+02	1.5060e+02
14	3.0948e+09	1.2081e+18	1.7610e+02	1.7610e+02
15	7.6294e+09	1.1648e+18	2.0364e+02	2.0364e+02
16	1.1529e+10	6.3970e+17	2.3326e+02	2.3326e+02
17	1.6697e+10	2.5407e+19	2.6493e+02	2.6493e+02
18	1.2177e+10	1.5944e+19	2.9868e+02	2.9868e+02
19	1.2095e+10	1.6505e+18	3.3450e+02	3.3450e+02
20	2.3078e+10	1.4312e+19	3.7241e+02	3.7241e+02
50	1.0690e+11	5.7653e+19	2.4870e+03	2.4870e+03
100	2.5607e+12	1.4665e+20	1.0268e+04	1.0268e+04
150	4.0301e+11	8.2715e+19	2.3413e+04	2.3413e+04
200	6.0479e+11	6.0850e+19	4.1942e+04	4.1943e+04

Tabela 3: Współczynnik uwarunkowania macierzy A_I oraz A_{II}

Co można wyczytać z powyższej tabeli? Istnieje prosta zasada, że, generalizując, współczynnik uwarunkowania macierzy rzędu 10^k oznacza utratę k cyfr znaczących wyniku. Zakładając więc, że liczba cyfr znaczących dla pojedynczej precyzji (`float32` w NumPy) wynosi 7, dla podwójnej (`float64`) wynosi natomiast 16, możemy wywnioskować, że współczynniki będące rzędu kolejno 10^7 oraz 10^{16} oznaczają fatalne uwarunkowanie macierzy, które generuje de facto losowe wartości, nie dając precyzji nawet jednej cyfry znaczącej.

Zauważmy, że powyższa zasada w pełni sprawdza się, współczynniki pokrywają się z oczekiwaniami. Dla pojedynczej precyzji i $n = 7$ (pierwszy zupełnie niemiernodajny wynik dla `float32`) mamy współczynnik rzędu 10^9 , natomiast dla $n = 13$ (pierwszy zupełnie niemiernodajny wynik dla `float64`) jest on 10^{17} .

Jeśli spojrzymy natomiast na tę część tabeli, która dotyczy A_{II} , obserwujemy tam powolny wzrost współczynnika uwarunkowania, który finalnie, dla $n = 250$, jest liczbą rzędu 10^4 , co wciąż daje nam przyzwoicie dokładny wynik nawet dla pojedynczej precyzji.

Zastanawiający może być fakt, że choć dla macierzy A_{Π} współczynnik uwarunkowania jest identyczny dla obu precyzji (`float32` oraz `float64`), to dla macierzy A_I nie jest to już prawdziwe. Przyczyny występowania tej rozbieżności między precyzjami jest precyzja samej arytmetyki zmiennoprzecinkowej. Zauważmy, że we wzorze na współczynnik uwarunkowania macierzy występuje macierz odwrotna A^{-1} . Wyznaczanie macierzy odwrotnej dla naszej macierzy A_I , która jest fatalnie uwarunkowana, wiąże się z ogromnymi błędami obliczeń numerycznych, więc również i sam współczynnik jest obciążony ogromnymi błędami. Nie sprawia to jednak, że obliczenia są zupełnie niemiarodajne, gdyż generalnie chodzi nam o rząd tego współczynnika (aby ocenić stratę cyfr znaczących), a więc możemy jasno pozostać przy wnioskach wysnutych w akapitach powyżej.

6. Eksperyment na macierzy A_{III}

Ostatni eksperyment tego laboratorium skupia się na innej tematyce. Operować będziemy na macierzy zdefiniowanej jako A_{III} we wstępie teoretycznym. Jest to szczególny typ macierzy, gdyż jest ona trójdzielna, tj. taka, która zawiera liczby niebędące zerem wyłącznie na trzech przekątnych: głównej, nad główną oraz pod główną. Z racji na ten szczególny rodzaj macierzy, możemy tutaj zastosować bardziej optymalny algorytm rozwiązywania układów równań, niż algorytm Gaussa.

Algorytm, którego użyjemy, to algorytm Thomasa. Ma on złożoność obliczeniową $O(n)$ podczas, gdy algorytm Gaussa ma złożoność $O(n^3)$. Celem tego eksperymentu jest rozwiązanie układu równań zadanego przy użyciu macierzy A_{III} przy użyciu obu algorytmów, a następnie porównanie dokładności uzyskanych wyników w zależności od metody, jak również czasu potrzebnego dla wykonania obliczeń oraz pamięci potrzebnej do działania algorytmu.

n	Eliminacja Gaussa		Algorytm Thomasa	
	f32	f64	f32	f64
2	0.0000e+00	0.0000e+00	0.0000e+00	2.2204e-16
3	0.0000e+00	0.0000e+00	5.9605e-08	2.2204e-16
4	0.0000e+00	2.2204e-16	5.9605e-08	2.2204e-16
5	0.0000e+00	2.2204e-16	5.9605e-08	2.2204e-16
6	0.0000e+00	2.2204e-16	1.1921e-07	2.2204e-16
7	1.1921e-07	2.2204e-16	1.1921e-07	2.2204e-16
8	1.1921e-07	2.2204e-16	1.1921e-07	2.2204e-16
9	1.1921e-07	2.2204e-16	1.1921e-07	2.2204e-16
10	1.1921e-07	2.2204e-16	1.1921e-07	2.2204e-16
11	1.1921e-07	2.2204e-16	1.1921e-07	2.2204e-16
12	1.1921e-07	2.2204e-16	1.1921e-07	2.2204e-16
13	1.1921e-07	2.2204e-16	1.1921e-07	2.2204e-16
14	1.1921e-07	2.2204e-16	1.1921e-07	2.2204e-16
15	1.1921e-07	2.2204e-16	1.1921e-07	2.2204e-16
16	1.1921e-07	2.2204e-16	1.1921e-07	2.2204e-16
17	1.1921e-07	2.2204e-16	1.1921e-07	2.2204e-16
18	1.1921e-07	2.2204e-16	1.1921e-07	2.2204e-16
19	1.1921e-07	2.2204e-16	1.1921e-07	2.2204e-16
20	1.1921e-07	2.2204e-16	1.1921e-07	2.2204e-16
50	1.1921e-07	2.2204e-16	1.1921e-07	2.2204e-16
100	1.1921e-07	2.2204e-16	1.1921e-07	2.2204e-16
150	1.1921e-07	2.2204e-16	1.7881e-07	2.2204e-16
200	1.1921e-07	2.2204e-16	1.7881e-07	2.2204e-16
250	1.1921e-07	2.2204e-16	1.7881e-07	4.4409e-16

Tabela 4: Porównanie błędu: Eliminacja Gaussa vs Algorytm Thomasa

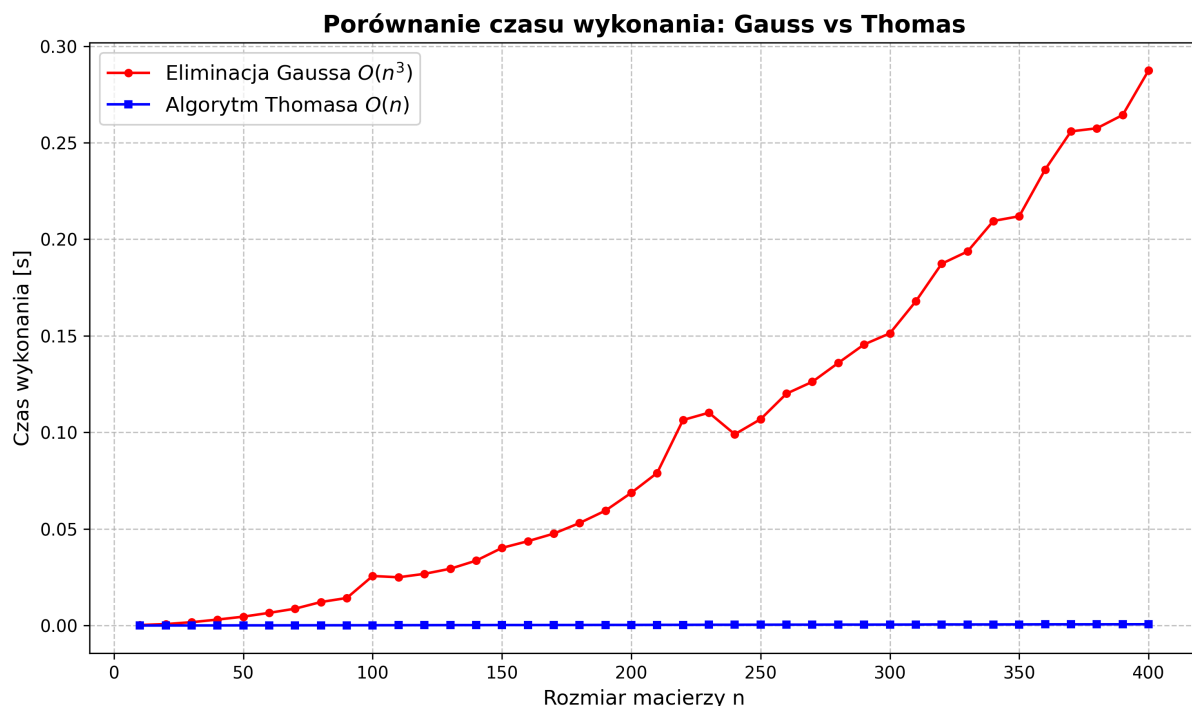
n	Eliminacja Gaussa				Algorytm Thomasa			
	Czas [s]		Pamięć [KB]		Czas [s]		Pamięć [KB]	
	f32	f64	f32	f64	f32	f64	f32	f64
2	7.2520e-04	4.9540e-04	1.8	1.8	1.7900e-04	1.6370e-04	0.7	0.7
3	5.9440e-04	3.4350e-04	1.9	2.0	2.3290e-04	1.3470e-04	0.7	0.7
4	4.3400e-04	4.1730e-04	2.0	1.9	1.5720e-04	1.4800e-04	0.7	0.8
5	5.8450e-04	6.6770e-04	1.9	2.0	1.8790e-04	1.8120e-04	0.7	0.8
6	7.8620e-04	8.6380e-04	1.9	2.1	2.1480e-04	2.1630e-04	0.7	0.9
7	9.5020e-04	9.9790e-04	2.0	2.2	2.4910e-04	2.5700e-04	0.8	0.9
8	1.3501e-03	1.2471e-03	2.0	2.4	2.8160e-04	2.8690e-04	0.8	0.9
9	1.5661e-03	1.4893e-03	2.1	2.5	3.2340e-04	3.2300e-04	0.8	1.0
10	1.9032e-03	1.8967e-03	2.2	2.7	4.3520e-04	3.8610e-04	0.8	1.0
11	2.3518e-03	2.2057e-03	2.3	2.9	4.2680e-04	4.8880e-04	0.8	1.0
12	2.6219e-03	2.4699e-03	2.4	3.1	4.5260e-04	5.6160e-04	0.9	1.1
13	1.1186e-02	6.1565e-03	2.5	3.3	9.8180e-04	9.2070e-04	0.9	1.1
14	6.9706e-03	4.8977e-03	2.6	3.6	9.5920e-04	5.2290e-04	0.9	1.2
15	3.7199e-03	6.9076e-03	2.8	3.8	5.5040e-04	1.0578e-03	0.9	1.2
16	7.1088e-03	4.2570e-03	2.9	4.1	6.0680e-04	5.6800e-04	1.0	1.3
17	4.6142e-03	4.6544e-03	3.0	4.4	6.3510e-04	6.0390e-04	1.0	1.3
18	5.0586e-03	5.0298e-03	3.2	4.7	6.5170e-04	6.2950e-04	1.0	1.3
19	5.5315e-03	5.6040e-03	3.3	5.0	7.0530e-04	7.6150e-04	1.0	1.4
20	6.1019e-03	6.1019e-03	3.5	5.3	7.0510e-04	8.5450e-04	1.0	1.4
50	3.5463e-02	3.5653e-02	12.1	22.4	1.7595e-03	1.7887e-03	1.6	2.6
100	1.4526e-01	1.4301e-01	41.9	82.2	3.4816e-03	3.5120e-03	2.6	4.5
150	3.1804e-01	3.1226e-01	91.3	181.0	5.3499e-03	5.2394e-03	3.6	6.5
200	5.5841e-01	5.5048e-01	160.3	318.9	7.0513e-03	6.9664e-03	4.6	8.5
250	8.6391e-01	8.8335e-01	248.8	495.8	8.7830e-03	9.2137e-03	5.5	10.4

Tabela 5: Porównanie zużycia zasobów: Eliminacja Gaussa vs Algorytm Thomasa

Najpierw przeanalizujemy Tabelę 4, która stanowi dane na temat błędów względem wektora referencyjnego X . Pokazuje ona, że bez względu na wybór algorytmu uzyskujemy podobną precyzję, tj. nasz błąd ma taki sam rząd.

Różnice między algorytmami uwidaczniają się dopiero, gdy weźmiemy na tapet dane prezentowane w Tabeli 5. Spójrzmy najpierw na czas potrzebny na znalezienie rozwiązania. Dla małych rozmiarów różnice są oczywiście marginalne, natomiast przy $n = 250$ (co wciąż nie daje jeszcze wielkiej macierzy) obserwujemy różnicę o dwa rzędy wielkości. Dla większych rozmiarów, zysk będzie jeszcze bardziej zauważalny, gdyż złożoność obliczeniowa algorytmu eliminacji Gaussa to $O(n^3)$, natomiast algorytm Thomasa pracuje w złożoności $O(n)$.

Wykres potrzebnego czasu w zależności od rozmiaru prezentuje Rysunek 3.



Rysunek 3: Porównanie czasu obliczeń algorytmów Gaussa oraz Thomasa.

Jeśli chodzi natomiast o pamięć, różnica jest również bardzo odczuwalna. Jeżeli spojrzymy na Tabelę 5, a konkretniej na kolumny dotyczące zużycia pamięci w KB dla obliczeń wykorzystujących podwójną precyzję, zauważyć możemy nawet blisko pięćdziesięciokrotny zysk pamięciowy na korzyść algorytmu Thomasa.

Dzieje się tak, gdyż wersja algorytmu Thomasa, którą zaimplementowałem w kodzie źródłowym i użyłem do obliczeń, operuje na samych przekątnych, nie potrzebujemy tworzyć całej macierzy A_{III} . Złożoność pamięciowa algorytmu Thomasa wynosi dzięki temu $O(n)$, podczas gdy algorytm Gaussa posiada złożoność pamięciową równą $O(n^2)$.

Warto jeszcze nadmienić, że od strony technicznej, do mierzenia czasu oraz zużycia pamięci zostały użyte funkcje bezpośrednio wbudowane w moduły Pythona, tj. `time.perf_counter()` dla pomiaru czasu, a także `tracemalloc.start()` i `tracemalloc.stop()` dla zużycia pamięci.

7. Podsumowanie

Na podstawie trzech eksperymentów na które składało się to laboratorium możemy dojść do kilku fundamentalnych wniosków na temat rozwiązywania układów równań. Przede wszystkim, należy mieć na uwadze wskaźnik uwarunkowania macierzy. Jest to cecha samej macierzy, nie metody. Wskaźnik uwarunkowania rzędu $\text{cond} = 10^k$ oznacza, że (generalizując) tracimy k cyfr znaczących z wyniku. Udało nam się to zaobserwować i udowodnić wykonując pierwsze dwa eksperymenty (ta sama metoda, różne uwarunkowanie macierzy A_I oraz A_{II}).

Drugi wniosek jest taki, że w przypadku, gdy macierz jest jakimś szczególnym przypadkiem, które często pojawiają się w obliczeniach (jak np. macierz trójdzielna), należy zbadać alternatywy dla generycznych metod rozwiązywania układów równań. W przypadku macierzy trójdzielnej udało nam się zbić złożoność obliczeniową z $O(n^3)$ do $O(n)$ oraz pamięciową z $O(n^2)$ do $O(n)$ poprzez zastosowanie algorytmu Thomasa.